

## W8: Açık Anahtarlı Kriptografi (Public Key Cryptography) ve Asimetrik Şifreleme

**1. Açık Anahtarlı Kriptografiye Giriş (Nedir?)** Açık anahtarlı kriptografi, şifreleme ve deşifreleme işlemleri için birbiriyle eşleşen **iki farklı anahtarın (Key Pair)** kullanıldığı kriptografik bir sistemdir.

- **Public Key (Açık/Genel Anahtar):** Herkese açık olarak paylaşılır, şifreleme yapmak veya imzaları doğrulamak için kullanılır.
- **Private Key (Özel/Gizli Anahtar):** Sadece sahibinde kalır ve kesinlikle gizli tutulur. Deşifreleme yapmak veya imza atmak için kullanılır.
- **Tarihsel Bağlam:** 1976'dan önce dünyada sadece tek anahtarlı simetrik şifreleme vardı. 1976'da Whitfield Diffie ve Martin Hellman bu konsepti (çift anahtar) dünyaya tanıttı. 1977'de ise MIT'den Rivest, Shamir ve Adleman ilk pratik algoritma olan RSA'yı icat etti.
- **Çalışma Mantiği:** Bu iki anahtar matematiksel olarak birbirine bağlıdır ancak en büyük esprî şudur: Bir kişinin *Public Key*'ine bakarak onun *Private Key*'ini hesaplamak/türetmek bilgisayarlar için hesapsal olarak imkânsızdır.
- **Analoji (Asma Kilit):** Public Key açık bir asma kilit gibidir. Herkes bu kilidi kapatarak içine veri koyabilir, ancak kilidi (veriyi) yalnızca anahtarın sahibi (Private Key) açabilir.

## 2. Simetrik ve Asimetrik Şifreleme Karşılaştırması

- **Simetrik Şifreleme (Geleneksel):** Şifreleme ve deşifreleme için **aynı tek bir anahtar** kullanılır. Büyük veriler için çok hızlı ve verimlidir. En büyük handikabı **"Anahtar Dağıtımı" (Key distribution) problemidir** (Ortak sırrı uzaktaki biriyle güvenlice nasıl paylaşacağız?). Ağda  $N$  kullanıcı varsa aralarında  $N(N - 1)/2$  adet anahtar üretilmesi gerekir ki bu çok zordur. (Örnekler: AES, DES, 3DES, ChaCha20).
- **Asimetrik Şifreleme (Public Key):** Public ve Private olmak üzere iki farklı anahtar kullanılır. Anahtar dağıtımı problemini anında çözer çünkü public key'i gizlemeye gerek yoktur.  $N$  kullanıcı için sadece  $2N$  anahtar üretmek yeterlidir. En büyük handikabı ise **simetrik şifrelemeye göre çok yavaş olmasıdır** (100-1000 kat arası). (Örnekler: RSA, ECC, ElGamal, DSA).
- **Hibrit Yaklaşım (Günümüzdeki Kullanım - TLS/SSL):** Sistemler aslında ikisini beraber kullanır. Yavaş ama güvenli olan Asimetrik şifreleme ile bir "oturum anahtarı" (simetrik anahtar) karşı tarafa güvenlice yollar. Ardından iletişim, hızlı olan Simetrik şifrelemeye geçerek devam eder.



### TRADITIONAL – SYMMETRIC

- ✓ Same key: encrypt & decrypt
- ✗ Key distribution is HARD
- ✗ N users  $\rightarrow$   $N(N-1)/2$  keys
- ✓ Fast processing



### PUBLIC KEY – ASYMMETRIC

- ✓ Two different keys: Public + Private
- ✓ Key distribution is EASY
- ✓ N users  $\rightarrow$  only 2N keys
- ✗ Slower than symmetric

## 3. Public Key Sistemlerinin 6 Temel Özelliği

Bir Asimetrik sistemin geçerli olabilmesi için şu 6 kuralı sağlaması gerekir:

1. Anahtar çiftlerini (Public ve Private) üretmek hesapsal olarak **kolay** olmalıdır.
2. Şifreleme yapmak **kolay** olmalıdır:  $C = E_{KU_b}(m)$  (Herkes public key ile kolayca veriyi şifreleyebilir).
3. Deşifreleme yapmak **kolay** olmalıdır:  $M = D_{KR_b}(C)$  (Private key sahibi veriyi kolayca çözebilir).
4. Bir saldırgan *Public Key* ve *Şifreli Metni (Ciphertext)* bilse dahi, mesajı deşifre etmesi **hesapsal olarak imkânsız** olmalıdır.
5. Bir saldırgan *Public Key*'i ( $KU_b$ ) bilse dahi, bundan *Private Key*'i ( $KR_b$ ) bulması **hesapsal olarak imkânsız** olmalıdır.
6. Şifreleme ve deşifreleme sırası **önemli değildir** (Commutative/Değişmeli özellik geçerlidir).

## 4. İki Farklı Çalışma Modu: Gizlilik (Secrecy) vs Kimlik Doğrulama (Authentication)

Açık anahtarlı kriptografi iki farklı temel amaç için iki farklı şekilde kullanılır:

-  **Gizlilik Modu (Secrecy / Confidentiality):**
  - **Amaç:** Sadece alıcının mesajı okuyabilmesini sağlamak.
  - **İşlem:** Alice, Bob'a mesaj atarken mesajı **Bob'un Public Key'i** ( $KU_b$ ) ile şifreler. (Formül:  $Y = E_{KU_b}(X)$ ).
  - **Sonuç:** Bu mesajı artık dünyada sadece Bob kendi Private Key'i ( $KR_b$ ) ile çözebilir. Mesaj gizli kalır.
-  **Kimlik Doğrulama Modu (Authentication / Digital Signature):**
  - **Amaç:** Mesajın kesinlikle kimden geldiğini kanıtlamak ve yolda değişmediğini garantilemek.
  - **İşlem:** Alice, mesajı **kendi Private Key'i** ( $KR_a$ ) ile şifreler (yani imzalar). (Formül:  $Y = E_{KR_a}(X)$ ).
  - **Sonuç:** Bu mesajı Alice'in Public Key'ini ( $KU_a$ ) kullanarak dünyadaki herkes çözebilir. Ancak mesajın Alice'in public key'i ile başarıyla çözülebilmesi, o mesajı **sadece**

Alice'in kendi private key'i ile hazırlayabileceğini kanıtlar. Buna "Dijital İmza" (Digital Signature) denir.

## Which Key? For What Purpose?

|                | 🔒 SECRECY / Confidentiality    | ✅ AUTHENTICATION                 |
|----------------|--------------------------------|----------------------------------|
| Encryption key | Bob's PUBLIC Key ( $K_{Ub}$ )  | Alice's PRIVATE Key ( $K_{Ra}$ ) |
| Decryption key | Bob's PRIVATE Key ( $K_{Rb}$ ) | Alice's PUBLIC Key ( $K_{Ua}$ )  |
| Guarantees     | Only recipient can read ✓      | Sender is verified ✓             |
| Formula        | $Y = E_{K_{Ub}}(X)$            | $Y = E_{K_{Ra}}(X)$              |

**5. Hem Gizlilik Hem Kimlik Doğrulamayı Birlikte Sağlamak (Super Formula) Sistemler** sadece gizlilik veya sadece doğrulama ile yetinmek zorunda değildir. Çift katmanlı şifreleme yapılabilir.

- **Şifreleme Sırası:** Alice önce mesajını **kendi private key'i** ile imzalar. Ardından çıkan sonucu **Bob'un public key'i** ile şifreler.
- **Formül:**  $Z = E_{K_{Ub}}[E_{K_{Ra}}(X)]$ .
- **Çözme Sırası (Bob'un Tarafı):** Bob önce **kendi private key'i** ile dış katmanı çözer. Sonra iç katmanın imzasını **Alice'in public key'i** ile doğrular.
- **Formül:**  $X = D_{K_{Ua}}[D_{K_{Rb}}(Z)]$ .

**6. Matematiksel Temeller ve Algoritmalar** Açık anahtarlı sistemlerin güvenliği, çözülmesi yüzlerce yıl süren zor matematik problemlerine dayanır.

- **RSA (Rivest-Shamir-Adleman):** "Büyük tamsayıları çarpanlarına ayırma" zorluğuna dayanır (Integer Factorization).  $p$  ve  $q$  adında iki devasa asal sayı seçilir. İkisinin çarpımı  $n$ 'i verir ( $n = p \times q$ ). Sistemi kırmak isteyen birinin, devasa  $n$  sayısının  $p$  ve  $q$  çarpanlarını bulması gerekir ki bu mevcut teknolojiyle çok zordur. (Günümüzde güvenli olması için en az 2048-bit, tercihen 3072-bit anahtar kullanılmalıdır).
- **ECC (Eliptik Eğri Kriptografisi):** Eliptik Eğri Ayrık Logaritma Problemine (ECDLP) dayanır. Formülü  $y^2 = x^3 + ax + b$ 'dir. En büyük avantajı: **RSA ile aynı seviyede güvenliği çok daha küçük anahtar boyutlarıyla sağlar!** (Örn: 256-bit bir ECC anahtarı, 3072-bit bir RSA anahtarı kadar güvenlidir). Hızlıdır ve IoT/Mobil cihazlar için standarttır (Örn: Bitcoin *secp256k1* kullanır).
- **Diffie-Hellman (DH):** Ayrık Logaritma problemine dayanır. İki tarafın güvensiz bir kanal üzerinden ortak bir "gizli anahtar" üretmesini sağlar. Ancak tek başına kimlik doğrulama yapmadığı için "Ortadaki Adam" (Man-in-the-Middle) saldırısına açıktır.

| Feature                     | RSA                    | ECC                | Diffie-Hellman     |
|-----------------------------|------------------------|--------------------|--------------------|
| Mathematical Basis          | Integer Factorization  | Elliptic Curve DLP | Discrete Logarithm |
| Key Size (128-bit security) | 3072 bits              | 256 bits           | 3072 bits          |
| Speed                       | Slow                   | Fast               | Moderate           |
| Use Cases                   | Encryption, Signatures | TLS, Mobile, IoT   | Key Exchange       |
| Quantum Vulnerable          | Yes                    | Yes                | Yes                |
| Standard                    | PKCS#1                 | NIST curves        | RFC 7919           |

## Asimetrik Algoritmaların Matematiksel İşleyişi ve Hesaplamaları

**1. Adım Adım RSA Algoritması (Anahtar Üretimi ve Hesaplama)** RSA, tamamen büyük asal sayıları çarpanlarına ayırmanın zorluğuna dayanır. Sınavda kağıt üzerinde adım adım yapman istenecek süreç tam olarak şöyledir:

- Anahtar Üretim Adımları (Key Generation):
  1. İki adet çok büyük rastgele asal sayı seçilir:  $p$  ve  $q$ .
  2. Modül değeri hesaplanır:  $n = p \times q$ . (Bu  $n$  değeri hem public hem private anahtarın bir parçasıdır) .
  3. Euler Totient fonksiyonu hesaplanır:  $\phi(n) = (p - 1)(q - 1)$ .
  4. Bir **Public Exponent** ( $e$ ) seçilir. Şart:  $e$  sayısı ile  $\phi(n)$  aralarında asal olmalıdır (  $\gcd(e, \phi(n)) = 1$ ). (Genellikle pratik sistemlerde 65537 seçilir) .
  5. **Private Exponent** ( $d$ ) hesaplanır. Şart:  $e \times d \equiv 1 \pmod{\phi(n)}$  olmalıdır. (Burada  $d$ 'yi bulmak için Genişletilmiş Öklid Algoritması kullanılır) .
  6. **Açık Anahtar (Public Key)**:  $(n, e)$  çiftidir.
  7. **Gizli Anahtar (Private Key)**:  $(n, d)$  çiftidir.
- Şifreleme (Encryption) Formülü:
  - $c = m^e \pmod{n}$  (Mesajın  $e$ . kuvveti alınır ve  $n$ 'e bölünerek kalan bulunur).
- Deşifreleme (Decryption) Formülü:
  - $m = c^d \pmod{n}$  (Şifreli metnin  $d$ . kuvveti alınır ve  $n$ 'e bölünerek orijinal mesaj bulunur).

🚩 SMALL EXAMPLE (for illustration only — real keys are 2048+ bits):

1.  $p = 61, q = 53$
2.  $n = 61 \times 53 = 3233$
3.  $\varphi(n) = 60 \times 52 = 3120$
4. Choose  $e = 17$  ( $\gcd(17, 3120) = 1$ )
5. Find  $d$ :  $17 \times d \equiv 1 \pmod{3120} \rightarrow d = 2753$

Public Key:  $(n=3233, e=17)$

Private Key:  $(n=3233, d=2753)$

🔒 ENCRYPT message  $m = 65$ :

$$c = 65^{17} \bmod 3233 = 2790$$

🔓 DECRYPT ciphertext  $c = 2790$ :

$$m = 2790^{2753} \bmod 3233 = 65 \quad \checkmark$$

**2. Sayısal İmza (Digital Signature) Mekanizması** Sayısal imzada süreç normal şifrelemenin **tam tersi** şeklinde işler. Amaç gizlilik değil, kimlik doğrulama ve bütünlüktür.

- İmzalama Süreci (Sign):
  1. Gönderici (Alice), mesajın hash değerini hesaplar:  $h = H(m)$  (Örn: SHA-256 ile).
  2. Bu hash değerini **kendi Private Key'i** ( $d$ ) ile şifreler (imzalar):  $s = h^d \pmod{n}$ .
  3. Mesajı ve imzayı birlikte gönderir:  $(m, s)$ .
- Doğrulama Süreci (Verify):
  1. Alıcı (Bob), gelen mesajın hash'ini tekrar hesaplar:  $h' = H(m)$ .
  2. Gelen  $s$  imzasını Göndericinin (Alice) **Public Key'i** ( $e$ ) ile çözer:  $h = s^e \pmod{n}$ .
  3. Eğer  $h == h'$  ise imza geçerlidir, mesaj yolda değişmemiştir!

**3. Diffie-Hellman Anahtar Değişimi (Key Exchange)** İki tarafın, güvenli olmayan bir hat üzerinden, dışarıdaki saldırganlara ifşa etmeden **ortak bir gizli simetrik anahtar (session key)** üretmesini sağlar. Ayrık Logaritma Problemine dayanır.

- Algoritmanın Adımları:
  1. Herkesin görebileceği iki halka açık parametre seçilir: Bir asal sayı  $p$  ve bir üreteç (generator)  $g$ .
  2. **Alice** gizli bir  $a$  sayısı seçer ve Bob'a  $A = g^a \pmod{p}$  değerini gönderir.
  3. **Bob** gizli bir  $b$  sayısı seçer ve Alice'in  $B = g^b \pmod{p}$  değerini gönderir.
  4. Alice ortak anahtarı hesaplar:  $s = B^a \pmod{p}$ .

5. Bob ortak anahtarı hesaplar:  $s = A^b \pmod{p}$ .
6. Matematiksel olarak her iki tarafta da  $s = g^{ab} \pmod{p}$  ortak sırrı oluşur.
7. **Zafiyeti:** Tek başına kimlik doğrulama yapamadığı için **Ortadaki Adam (Man-in-the-Middle)** saldırılarına karşı savunmasızdır.

#### 4. ECC (Eliptik Eğri) ve ElGamal Mantiğı

- **ElGamal:** Güvenliği tamamen Diffie-Hellman gibi Ayrık Logaritma problemine ( $g^x \pmod{p}$  değerinden  $x$ 'i bulma zorluğu) dayanır.
- **ECC (Elliptic Curve Cryptography):**  $y^2 = x^3 + ax + b \pmod{p}$  eğrisi üzerindeki noktaların çarpımı problemine dayanır. **En büyük avantajı:** Çok daha küçük anahtar boyutlarıyla devasa güvenlik sunmasıdır. Örneğin, 256-bit ECC, 3072-bit RSA ile aynı güçtedir. Bu yüzden mobil cihazlar ve nesnelerin interneti (IoT) için vazgeçilmezdir.

#### 7. Gerçek Hayatta Nerede Kullanılır? (PKI ve TLS)

- **PKI (Public Key Infrastructure):** İnternetin güven temelidir. Certificate Authority (CA - Sertifika Otoritesi) isimli yapılar sertifika dağıtır. Bir X.509 sertifikası, bir kurumun gerçek kimliği ile Public Key'ini birbirine mühürler.
- **HTTPS ve TLS Handshake:** Tarayıcınla bir siteye (örn: banka) girdiğinde, sunucu sana sertifikasını (içinde Public Key var) yollar. Tarayıcın bu public key'i kullanarak bir "oturum anahtarı" (pre-master secret) üretilip şifreler ve sunucuya yollar. Sadece sunucu kendi private key'i ile bunu açabilir. Anahtarlar eşleşince, hızlı olan Simetrik iletişime geçilir.
- **Diğer Örnekler:** E-posta imzalama (PGP, S/MIME), Kripto paralar (İşlemler Private key ile imzalanıp blockchain ağına Public key ile kanıtlanır), SSH Login (Public key'ini sunucuya önceden atarsın, bağlanırken Private key'in ile "ben oyum" dersin).

#### 8. Açık Anahtarlı Sistemlere Karşı Saldırıları ve Gelecek

- **Yan Kanal (Side-Channel) Saldırıları:** Algoritmanın matematiğini kırmak yerine, makinenin çalışırken harcadığı zamanı (Timing attacks) veya harcadığı elektriği (Power analysis) izleyerek gizli anahtarı bulmaya çalışmaktır.
- **Kuantum Tehdidi ve PQC:** Kuantum bilgisayarları "Shor'un Algoritması"nı kullanarak saatler içinde koca bir RSA'yı (2048-bit) veya ECC'yi kırabilecek güce ulaşacaktır. Bu yüzden NIST (2024 yılında) kuantum bilgisayarlarının bile kıramayacağı **Post-Quantum Cryptography (PQC)** standartlarını yayınlamıştır (Örn: CRYSTALS-Kyber, CRYSTALS-Dilithium). Mevcut sistemlerin 2030-2035 gibi tamamen bu yeni algoritmalara geçmesi beklenmektedir.

## W9: Kriptografik Hash Fonksiyonları ve Veri Bütünlüğü

**1. Güvenliğin Temel Taşları ve Hash'in Rolü** Sadece şifreleme (Encryption) yapmak güvenliğini sağlamak için yeterli değildir. Şifrelenmiş bir veri gizli kalabilir ancak iletim sırasında yetkisiz kişilerce değiştirilip değiştirilmediğini veya göndericinin gerçekten iddia ettiği kişi olup olmadığını şifreleme tek başına bilemez. Güvenliğin üzerine inşa edildiği 3 temel taş şöyledir:

- **Gizlilik (Confidentiality):** Şifreleme (Encryption) ile sağlanır.
- **Bütünlük (Integrity):** Hash fonksiyonları ile sağlanır.
- **Kimlik Doğrulama (Authentication):** MAC (Message Authentication Code) veya Dijital İmzalar ile sağlanır.

**2. Hash Fonksiyonu Nedir?** Hash fonksiyonları, herhangi bir uzunluktaki (değişken) girdiyi veya metni alıp, sabit uzunlukta benzersiz bir "özet" (digest/hash değeri) üreten matematiksel işlemlerdir.

- **Sabit Çıktı Uzunluğu:** Girdi ister tek bir harf, ister ansiklopedi uzunluğunda olsun, çıktı her zaman aynı uzunluktadır.
- **Hızlı ve Deterministik:** İşlem hızlı gerçekleşir ve aynı girdi her zaman birebir aynı hash değerini (çıktıyı) üretir.
- **Tek Yönlü (One-Way):** Üretilen hash değerine bakarak orijinal metni (girdiyi) geri elde etmek neredeyse imkânsızdır.
- **Dijital Parmak İzi:** Tıpkı insanların parmak izi gibi, her mesajın kendine özgü, eşsiz bir hash değeri vardır.
- **Çığ Etkisi (Avalanche Effect):** Girdide yapılacak en ufak bir değişiklik (örneğin tek bir harfin veya noktanın değişmesi), üretilen hash değerinin tamamen (büyük ölçüde) değişmesine neden olur.

## Avalanche Effect (SHA-256)

Input 1: "hello"

Input 2: "hello!"

Hash 1:

2cf24dba5fb0a30e26e83b2ac5b9e29e  
1b161e5c1fa7425e73043362938b9824

Hash 2:

ce06092fb948d9ffac7d1a376e404b26  
b7575bcc11ee05a4615fef4fec3a308b

## 3. İyi Bir Hash Fonksiyonunun 3 Temel Özelliği

Bir hash fonksiyonunun güvenli kabul edilebilmesi için şu üç dayanıklılığa sahip olması şarttır:

- **1. Preimage Resistant (Tek Yönlü Olma):** Hash değerini (çıktıyı) bilsen bile, bu değeri üreten orijinal mesajı bulmak imkânsız olmalıdır. (Metafor: Çırpılmış bir yumurtayı eski haline döndüremezsin).

- **2. Second Preimage Resistant (Zayıf Çarpışma Dayanıklılığı):** Elinde orijinal bir mesaj ve onun hash değeri varken, **birebir aynı hash değerini üreten "farklı" bir mesaj** yaratmak imkânsız olmalıdır. (Metafor: Bir belgeyi veya sözleşmeyi taklit edemezsin).
- **3. Collision Resistant (Güçlü Çarpışma Dayanıklılığı):** Aynı hash değerini üretecek **herhangi iki farklı mesaj** bulmak imkânsız olmalıdır. (Metafor: Dünyada iki farklı kişinin aynı parmak izine sahip olmaması).

#### 4. Hash ve Şifreleme (Encryption) Arasındaki Farklar

- **Şifreleme (Encryption):** İki yönlüdür (Şifrele ve Çöz). İşlemler için bir Anahtar (Key) gerekir. Temel amacı verinin gizliliğini korumaktır. (Örn: AES, RSA algoritmaları).
- **Hash:** Tek yönlüdür (Sadece ileri gidilir, geri dönüş yoktur). Çalışmak için bir anahtara ihtiyaç duymaz (anahtarsızdır). Temel amacı verinin bütünlüğünü doğrulamaktır. (Örn: MD5, SHA-256).



#### 5. Hash Fonksiyonlarının Kullanım Alanları

- **Şifre Saklama:** Veri tabanları kullanıcıların şifrelerini düz metin olarak değil, hash'lenmiş olarak saklar. Sistem çalınsa bile saldırgan gerçek şifreyi göremez.
- **Dosya Bütünlüğü:** İndirilen bir dosyanın (örn: MD5 checksum) yolda bozulup bozulmadığını kontrol eder.
- **Dijital İmza:** Bir belgenin hash değeri, özel bir anahtarla şifrelenerek dijital imza oluşturulur ve alıcı bunu doğrular.
- **HMAC:** Hash işlemi ve gizli bir anahtar birleştirilerek hem mesaj bütünlüğü hem de kimlik doğrulaması sağlanır.
- **Saldırı Tespiti:** Sistem dosyalarının hash değerleri saklanır, dosyada bir değişiklik (saldırı) olursa hash değişeceği için hemen fark edilir.
- **Blockchain:** Her bir blok, kendinden bir önceki bloğun hash değerini içerir, bu da zincirin değiştirilemez olmasını sağlar. SHA-256 algoritması yaygın olarak kullanılır.

## 6. Önemli Hash Algoritmaları (MD5 ve SHA)

- **MD Ailesi (Message Digest):** Ronald Rivest tarafından tasarlanmıştır.
  - MD2 (1989): 8-bit bilgisayarlar içindir.
  - MD4 (1990): Daha hızlı ama güvenlik açısından zayıftır.
  - **MD5 (1992):** RFC 1321 standardıdır. **Teknik özellikleri:** 128-bit hash çıktısı verir, veriyi 512-bitlik bloklar halinde işler, 64 adımda çalışır (4 tur x 16 adım) ve Küçük-Endian (Byte sıralaması) yapısını kullanır.
  - *MD5'in Çöküşü:* 1997'de çarpışma yaratmak teorik olarak kolaylaştı, 2004'te pratik olarak çarpışma saldırısı gösterildi ve 2010'da US-CERT kullanımı tamamen yasakladı. Artık güvenli değildir!.
- **SHA Ailesi (Secure Hash Algorithm):** NIST tarafından tasarlanmıştır. 512 bit blok boyutu kullanırlar (SHA-384 ve SHA-512 hariç, onlar 1024 bit kullanır).
  - SHA-0 (160 bit) ve SHA-1 (160 bit): Kırılmıştır ve zayıftır, aşamalı olarak kullanımdan kaldırılmaktadır.
  - **SHA-2:** SHA-224, SHA-256, SHA-384 ve SHA-512 versiyonlarını barındırır. Günümüzde hala **güvenli kabul edilir**.
  - **SHA-3 (Keccak):** Yeni nesil standarttır. MD5 tamamen kırıldığı, SHA-1'de zafiyetler bulunduğu ve SHA-2'nin yapısı SHA-1'e çok benzediği için (endişe yarattığı için) NIST yeni bir yarışma başlattı (2007-2012). 5 finalist (BLAKE, Grøstl, JH, Keccak, Skein) arasından **Keccak** kazanmıştır. Diğerlerinden farklı olarak tamamen yeni bir mimari olan **"sponge construction"** yapısına sahiptir.

| Algoritma | Hash Uzunluğu | Blok Boyutu | Adım Sayısı | Durum   |
|-----------|---------------|-------------|-------------|---------|
| SHA-0     | 160 bit       | 512 bit     | 64          | Kırıldı |
| SHA-1     | 160 bit       | 512 bit     | 80          | Zayıf   |
| SHA-224   | 224 bit       | 512 bit     | 64          | Güvenli |
| SHA-256   | 256 bit       | 512 bit     | 64          | Güvenli |
| SHA-384   | 384 bit       | 1024 bit    | 80          | Güvenli |
| SHA-512   | 512 bit       | 1024 bit    | 80          | Güvenli |

## 7. Hash Fonksiyonlarına Yapılan Saldırıları

- **Kaba Kuvvet (Brute Force):** Tüm olası mesajları tek tek deneme yöntemidir. Birinci preimage saldırısı için hash uzunluğu  $n$  ise,  $2^n$  deneme gerektirir. (Örn: 128-bit hash için  $2^{128}$  ki bu da devasa bir sayıdır).

- **Kriptanaliz:** Algoritmanın kendi içindeki matematiksel zayıflıklarını (açıklarını) arar. Kaba kuvvetten çok daha hızlıdır ve MD5 ile SHA-0 algoritmaları bu şekilde kırılmıştır.
- **Doğum Günü Saldırısı (Birthday Attack):** "Herhangi iki" farklı mesajın aynı hash'i vermesini (çarşıma) hedefler. Kaba kuvvete göre çok tehlikelidir çünkü sadece  $2^{(n/2)}$  deneme yeterlidir!. (Örn: 128-bitlik bir hash için  $2^{128}$  yerine sadece  $2^{64}$  deneme ile çarşıma bulunabilir).

## 8. Hash Bütünlük İçin Yeterli Mi? (MAC ve HMAC Devreye Giriyor)

- **Problem:** Sadece normal bir Hash fonksiyonu kullanıldığında sistem tamamen güvende olmaz. Bir saldırgan iletim sırasında sadece mesajı değiştirmekle kalmaz, değiştirdiği mesajın **yeni hash değerini de hesaplayıp** alıcıya yollayabilir. Bu durumda alıcı, mesajın değiştirildiğini anlayamaz (Bütünlük bozulur).
- **Çözüm (MAC - Message Authentication Code):**  $MAC = f(K, m)$  formülüyle çalışır. Sistemin içine sadece iki tarafın bildiği bir "gizli anahtar" (Secret Key) dahil edilir. Hem bütünlük (integrity) hem de kimlik doğrulama (authentication) sağlar.
- **HMAC:** Hash fonksiyonu ile Gizli Anahtarın bir arada kullanıldığı MAC türüdür. Günümüzde yaygın ve son derece güvenlidir.

## Hash vs HMAC

| Feature        | Hash | HMAC   |
|----------------|------|--------|
| Key            | No   | Yes    |
| Integrity      | Weak | Strong |
| Authentication | No   | Yes    |
| Secure alone   | No   | Yes    |

## 9. Entropi (Entropy) ve Rastgelelik

- **Entropi**, sistemdeki "belirsizliği" (uncertainty) temsil eder.
- Kriptografide güçlü bir sistem kurabilmek (özellikle anahtar üretimi - key generation aşamalarında) için entropi hayati öneme sahiptir.
- Doğası gereği Kaos sistemleri (chaos systems) ile bağlantılıdır.

## W10: PKI, Sertifikalar ve TLS/HTTPS Güvenliği

## 1. Güven Problemi ve Public Key Kime Ait?

- **Senaryo:** Bir web sitesine (örneğin bankaya) girdiğimizde sunucu bize şifreleme yapabilmemiz için kendi "Public Key"ini gönderir. Ancak araya giren bir saldırgan (Man-in-the-Middle) bankanın anahtarı yerine **kendi sahte public key'ini** bize yollayabilir.
- **Sorun:** Bizim tarayıcımız, kendisine gelen bu public key'in gerçekten bankaya mı ait olduğunu yoksa bir saldırgana mı ait olduğunu tek başına bilemez.

## 2. Çözüm: Certificate Authority (CA - Sertifika Otoritesi)

- Tıpkı gerçek hayatta kimliğimizi doğrulayan bir noter gibi çalışan, internetin **güvenilir üçüncü taraf kuruluşlarıdır** (Örn: DigiCert, Let's Encrypt, Comodo).
- Tarayıcılar ve işletim sistemleri bu büyük CA şirketlerinin public key'lerini önceden tanır ve içlerinde yüklü olarak barındırır.
- **Nasıl Çalışır?:** Site sahibi bir CSR (Certificate Signing Request) hazırlar, CA bu kişinin gerçekten o sitenin sahibi olup olmadığını doğrular ve ardından sertifikayı **kendi private key'i ile imzalayarak** onaylar .

## 3. X.509 Dijital Sertifika Anatomisi (İçinde Neler Var?) Dünya standardı olan X.509 dijital sertifikasının içinde şu kritik bilgiler yer alır:

- **Subject (Konu):** Sertifikanın kime/hangi domaine ait olduğunu belirtir (Örn: CN=[www.banka.com](http://www.banka.com)).
- **Public Key:** Sitenin şifrelemede kullanılacak asıl açık anahtarıdır (Örn: RSA 2048-bit).
- **Issuer (Veren):** Sertifikayı onaylayıp imzalayan CA'nın adıdır (Örn: DigiCert).
- **Geçerlilik (Validity):** Sertifikanın başlangıç ve son kullanma tarihidir.
- **CA İmzası (En Kritik Alan):** CA'nın tüm sertifikanın bütünlüğünü korumak için **kendi private key'i** ile attığı imzadır (Örn: SHA256withRSA). Kimse bu imzayı taklit edemez.

| X.509 Dijital Sertifika |                                            | Subject                                                                                                                |
|-------------------------|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Subject (Konu):         | CN=www.banka.com<br>O=Banka A.Ş., C=TR     | Sertifikanın hangi web sitesine ait olduğunu söyler. Tarayıcı, gittiğin URL ile Subject'teki CN alanını karşılaştırır. |
| Public Key:             | RSA 2048-bit<br>30 82 01 0a 02 82 01 01... | Public Key                                                                                                             |
| Issuer (Veren):         | CN=DigiCert Global CA<br>O=DigiCert Inc    | Sitenin public key'i burada saklanır. İstemci bunu kullanarak şifreli iletişim kurar.                                  |
| Geçerlilik:             | 2024-01-01 →<br>2025-01-01                 | Issuer                                                                                                                 |
| Seri No:                | 04:F1:7A:2B:...                            | Sertifikayı kim imzaladı? CA'nın ismi buradadır. Tarayıcı bu CA'ya güvenip güvenmediğini kontrol eder.                 |
| CA İmzası:              | SHA256withRSA<br>3D 88 A1 9C:...           | CA İmzası                                                                                                              |
|                         |                                            | En kritik alan! CA kendi private key'i ile tüm sertifikayı imzalar. Kimse bu imzayı taklit edemez.                     |

**4. Sertifika Zinciri (Chain of Trust)** Sistem tek bir yerden değil, güvenlik amaçlı hiyerarşik bir güven zincirinden oluşur:

- **Root CA (Kök CA):** Her şeyin başladığı zirvedir. Kendi kendini imzalar (self-signed) ve çalınmaması için genelde internete bağlı değildir (offline tutulur).
- **Intermediate CA (Ara CA):** Root CA tarafından yetkilendirilen alt CA'dır. Günlük rutin imzalama işlemlerini yapar, böylece Root CA'yı riske atmaktan korur.
- **End-Entity Certificate (Sunucu):** Girdiğimiz sitenin (Banka) sertifikasıdır, Intermediate CA tarafından imzalanmıştır .



**5. Neden Sadece RSA Kullanmıyoruz? (Hibrit TLS Yaklaşımı)** Asimetrik şifreleme (RSA) ne kadar güvenli olursa olsun, tüm internet trafiğini onunla şifrelemek imkansızdır.

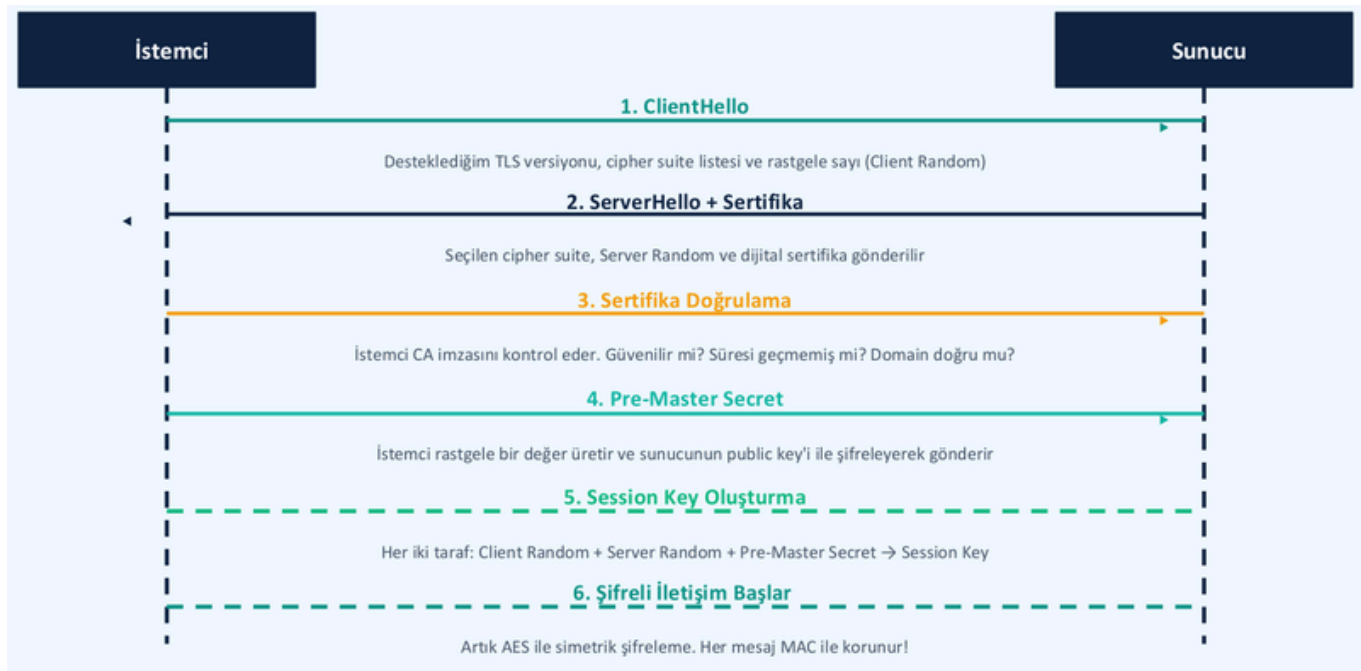
- **Çok Yavaş:** AES gibi simetrik algoritmalarla göre 100-1000 kat yavaştır (Film izlenemez).
- **Küçük Veri Problemi:** Sadece ufak bloklar halinde çalışır.

- **Forward Secrecy Yoktur:** Eğer Private key ileride çalınırsa, kaydedilmiş tüm geçmiş konuşmalar çözülebilir!.
- **Çözüm (TLS Hibrit Yapı):** Yavaş olan RSA, sadece "Session Key" (Oturum Anahtarı) denen küçük bir anahtar güvenli şekilde karşıya iletmek için kullanılır. Sonrasında asıl büyük veri trafiği çok daha hızlı olan **AES (Simetrik)** ile şifrelenir. Mesajların bütünlüğü ise **HMAC** ile doğrulanır.

## 6. TLS Handshake (El Sıkışma) Adım Adım Nasıl Gerçekleşir?

Bir siteye bağlandığında arka planda sırasıyla şunlar olur:

1. **ClientHello:** Tarayıcı, desteklediği TLS versiyonlarını ve algoritmaları (cipher suites) sunucuya bildirir. Ayrıca kendi ürettiği rastgele sayıyı (**Client Random**) gönderir.
2. **ServerHello + Sertifika:** Sunucu listeden bir algoritma seçer, kendi rastgele sayısını (**Server Random**) ve X.509 Dijital Sertifikasını tarayıcıya yollar.
3. **Sertifika Doğrulama (Kilit İşareti):** Tarayıcı sertifikayı inceler. *CA güvenilir mi? Süresi geçerli mi? Domain doğru mu? İmza geçerli mi? Zincir tam mı?*
4. **Pre-Master Secret:** Her şey tamamsa istemci yeni bir rastgele değer (Pre-Master Secret) üretir ve bunu **Sunucunun Public Key'i ile şifreleyerek** sunucuya atar.
5. **Session Key Üretimi:** Her iki taraf da ellerindeki üç veriyi (Client Random + Server Random + Pre-Master Secret) özel bir PRF fonksiyonuna sokarak **Session Key (Oturum Anahtarı)** üretir.
6. **Şifreli İletişim Başlar:** Asimetrik (RSA) kısım biter. Artık taraflar sadece kendi bildikleri bu Session Key ile yüksek hızlı AES simetrik şifrelemesine başlar. Bağlantı kopunca Session Key tamamen silinir.



## 7. TLS Hangi Saldırıları Önler?

- **Man-in-the-Middle (MITM):** CA imzalı sertifikalar sayesinde saldırgan araya girip kendi sertifikasını sunamaz, sunarsa tarayıcı anında kırmızı uyarı verir.
- **Eavesdropping (Ağı Dinleme):** Tüm veri AES ile şifrelendiği için Wireshark vb. araçlarla ağı dinleyen biri sadece anlamsız şifreli metinler (cipher text) görür.
- **Replay Attack (Tekrarlama):** Her oturumda yepyeni bir Session Key ve özel sıra numaraları kullanıldığı için saldırgan çaldığı eski bir paketi tekrar yollayamaz.
- **Data Tampering (Veri Değiştirme):** Her pakete HMAC eklendiği için yolda paketin tek bir biti bile değiştirilse alıcı bunu anlar ve paketi reddeder.

## RSA'in Sınırları ve Session Key Üretimi

**Neden Tüm İnternet Trafiğini Sadece RSA ile Şifrelemiyoruz?** Mantıklı bir soru: Asimetrik şifreleme (RSA) zaten güvenliyse, neden tüm mesajları baştan sona RSA ile şifrelemiyoruz? Sadece RSA kullansaydık şu devasa sorunlarla karşılaşırđık:

- 🐢 **Çok Yavaş Olması:** RSA algoritması, karmaşık matematiksel (büyük asal sayılar) işlemler yaptığı için AES gibi simetrik algoritmalara göre 100 ile 1000 kat arası daha yavaştır. Tüm veriyi RSA ile şifreleseydik, Netflix'ten bir film izlemek veya büyük bir dosya indirmek saatlerce sürerdi.
- 📦 **Büyük Veri Problemi (Overhead):** RSA mimarisi geređi verileri çok küçük bloklar halinde şifrelemek zorundadır. İnternetteki her bir veri paketini ayrı ayrı küçük RSA bloklarına bölüp şifrelemek sistemde inanılmaz bir yük (overhead) yaratır.
- 🔑 **Forward Secrecy (İleri Yönlü Gizlilik) Eksikliği:** RSA'da sunucunun sabit bir Private Key'i vardır. Eğer bir saldırgan şifreli trafiđi yıllarca kaydeder ve bir gün bu Private Key'i çalmayı başırırsa, geçmişte kaydedilmiş tüm iletişimler geriye dönük olarak çözülebilir.
- ✅ **Çözüm: Hibrit Yaklaşım (TLS):** Bu yüzden TLS protokolü zekice bir hibrit yöntem kullanır. Ağır ve yavaş olan RSA, sadece küçücük bir veri olan "Session Key"i karşıya güvenle iletmek (anahtar deđiřimi) için kullanılır. Sonrasında asıl büyük veri trafiđi (film, bankacılık verisi vs.) hızlı olan AES ile şifrelenir. Verilerin deđiřtirilip deđiřtirilmediđi ise HMAC ile korunur.

**Session Key (Oturum Anahtarı) Nasıl Üretilir?** TLS el sıkışması (handshake) sırasında iki tarafın ortak, geçici ve son derece güvenli bir "Session Key" üretmesi gerekir. Bu üretim tek bir tarafın inisiyatifine bırakılmaz, üç farklı rastgele veri birleřtirilerek elde edilir:

1. **Client Random:** İstemcinin (Tarayıcı) iletişim başında kendi ürettiđi rastgele sayıdır.
  2. **Server Random:** Sunucunun iletişim başında kendi ürettiđi rastgele sayıdır.
  3. **Pre-Master Secret:** İstemcinin ürettiđi ve araya kimse giremesin diye **Sunucunun Public Key'i (RSA) ile şifreleyerek** gönderdiđi son derece gizli bir deđerdir.
- **Üretim Formülü:** Her iki taraf da bu üç deđere (Client Random + Server Random + Pre-Master Secret) sahip olduktan sonra, bunları **Pseudo-Random Function (PRF)** adı verilen

matematiksel bir fonksiyona sokarlar. Bu işlemin sonucunda AES şifrelemesinde kullanılacak olan nihai **Session Key** ortaya çıkar.



### Session Key Ne İşe Yarar ve Akıbeti Ne Olur?

- **AES Şifreleme:** İletişim boyunca tüm web trafiği bu Session Key kullanılarak AES-256 ile şifrelenir.
- **MAC/HMAC Üretimi:** Veri bütünlüğünü doğrulamak (imzalamak) için her mesaja eklenecek olan MAC değerleri de yine bu Session Key'den türetilir.
- **Oturum Sonu (Forward Secrecy):** Web sitesi sekmesini kapattığında (bağlantı koptuğunda), her iki taraftaki Session Key kalıcı olarak **silinir** ve bir daha asla tekrar kullanılmaz. Sonraki her bağlantıda yepyeni bir Session Key üretilir, böylece geçmiş bağlantılar sonsuza dek güvende kalır.

### 8. Forward Secrecy (İleri Yönlü Gizlilik) ve Yeni Nesil TLS

- **Problem:** Eski sistemlerde, saldırgan şifreli trafiği yıllarca kaydederse ve bir gün sunucunun Private Key'ini çalarsa, geçmişteki tüm konuşmaları çözebilirdi.
- **Çözüm:** Forward Secrecy, her oturum için geçici (ephemeral) anahtarlar üretir. Oturum bittiğinde bu anahtarlar imha edilir. Böylece ileride private key çalınsa bile **geçmiş veriler sonsuza dek güvende kalır**.
- **Önemli Not:** Modern **TLS 1.3** protokolü RSA ile anahtar değişimini tamamen kaldırmıştır. Günümüzde Forward Secrecy garantisi sunan **ECDHE (Elliptic Curve Diffie-Hellman Ephemeral)** kullanımı zorunludur.

## Büyük Resim: Her Şey Birlikte Nasıl Çalışır?



## Aklınızda Kalsın!

Bu dersten 5 şey götürün:

- 1 Public key tek başına güven vermez**  
CA imzası olmadan kime ait olduğu bilinmez.
- 2 Dijital sertifika = Public key + Kimlik + CA İmzası**  
X.509 formatı dünyada standart.
- 3 TLS hibrit şifreleme kullanır**  
RSA ile anahtar, AES ile veri şifrelenir.
- 4 Her oturum farklı session key**  
Forward secrecy sayesinde geçmiş güvende.
- 5 Tarayıcı 5 şeyi kontrol eder**  
CA, Geçerlilik, Domain, İmza, Zincir.

## W11: KAOTİK KRİPTOGRAFİ (CHAOTIC CRYPTOGRAPHY)

**Ana Tez:** Rastgele görünen bir çıktı (random-looking output), kriptografik olarak güvenli bir sistem anlamına gelmez.

### 1. GEÇMİŞ DERSLERİN ÖZETİ VE GİRİŞ (WHERE WE LEFT OFF)

Modern internet güvenliği (TLS), farklı kriptografik bileşenlerin bir arada çalıştığı **hibrit bir mimari** üzerine kuruludur:

- Anahtar Değişimi (Key Exchange):** RSA veya ECDH algoritmaları ile güvenli olmayan bir kanal üzerinden ortak bir gizli anahtar belirlenir.

- **Veri Şifreleme (Data Encryption):** AES-256 gibi simetrik blok şifreleme algoritmaları ile ana veri şifrelenir.
- **Bütünlük Kontrolü (Integrity):** HMAC kullanılarak verinin yolda değiştirilip değiştirilmediği doğrulanır.
- **Kimlik Doğrulama (Identity / PKI):** X.509 dijital sertifikaları ile tarafların kimliği doğrulanır.

**Temel Soru:** Kaotik sistemler, rüştünü ispatlamış bu modern kriptografik ilkelere (primitives) alternatif veya destek olabilir mi?

## 2. KAOS NEDİR? KAOTİK SİSTEMLERİN ÜÇ TEMEL SÜTUNU

Bir sistemin "kaotik" olarak adlandırılabilmesi için doğrusal olmayan (non-linear) dinamik denklemlere sahip olması ve şu üç temel özelliği barındırması gerekir:

1. **Deterministik (Belirlenimci) Yapı:** Sistem tamamen kurallıdır ve rastgele (stokastik) değildir. Aynı başlangıç değerleri ve parametreler girildiğinde, sistem her zaman birebir aynı çıktı dizisini üretir. Denklem formasyonu:  $x_{n+1} = f(x_n)$  şeklindedir.
2. **Başlangıç Koşullarına Aşırı Duyarlılık (Sensitivity to Initial Conditions):** "Kelebek Etkisi" (Butterfly Effect) olarak da bilinir. Başlangıç değerindeki ihmal edilebilir bir fark (örneğin milyonda bir), sadece birkaç iterasyon (adım) sonra sistemin tamamen farklı bir yörüngeye savrulmasına neden olur.
3. **Doğrusal Olmama (Non-linearity):** Sistem bileşenleri arasında doğrusal bir orantı yoktur. Girdide yapılan küçük bir değişiklik, çıktıda katlanarak büyüyen ve tahmin edilemeyen sonuçlar doğurur. Denklemler mutlaka doğrusal olmayan çarpımlar veya üstler içerir.

**Kriptografik Çıkarım:** Bir sistem matematiksel olarak %100 deterministik olmasına rağmen, dışarıdan bakan bir gözlemci için tamamen rastgele (random) görünebilir.

# Three Pillars of Chaotic Systems



## DETERMINISTIC

The system is NOT random. Given the same initial conditions and parameters, it always produces the same output.

$$x_{n+1} = f(x_n)$$



## SENSITIVE

Tiny differences in starting values lead to wildly different outputs after just a few iterations. The 'butterfly effect'.

$$x_0 = 0.500000 \text{ vs } 0.500001$$



## NON-LINEAR

Chaotic systems involve non-linear equations — a small input change does not cause a proportional output change.

$$x_{n+1} = r \cdot x_n \cdot (1 - x_n)$$

*Key insight: A system can be deterministic AND appear random at the same time.*

## Sayısal Örneklerle Kelebek Etkisi (The Butterfly Effect in Numbers)

İki farklı sistem tasarlandığını ve başlangıç değerleri arasında sadece  $\Delta = 0.000001$  kadar bir fark olduğunu varsayalım:

- **Sistem A başlangıcı:**  $x_0 = 0.500000$
- **Sistem B başlangıcı:**  $x_0 = 0.500001$

İterasyonlar ilerledikçe iki sistem arasındaki fark ( $\Delta$ ) şu şekilde büyür:

- $n = 1$  için:  $\Delta \approx 0.000002$  ( $2 \times 10^{-6}$ )
- $n = 5$  için:  $\Delta \approx 0.0002$  ( $2 \times 10^{-4}$ )
- $n = 10$  için:  $\Delta \approx 0.05$  (Yörüngeler görünür şekilde ayrışır)
- $n = 20$  için:  $\Delta \approx 0.70$  (İki sistem tamamen bağımsız ve alakasız davranır)

## 3. LOJİSTİK HARİTA (THE LOGISTIC MAP)

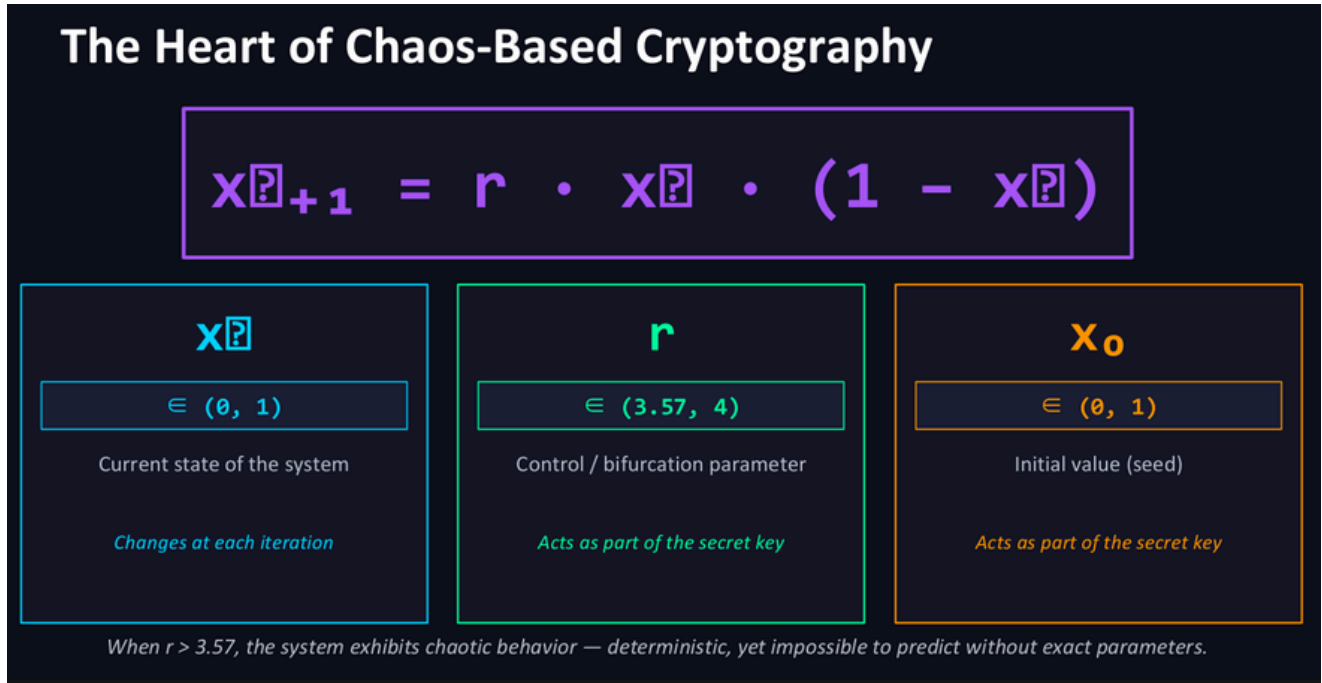
Kaos tabanlı kriptografide en sık kullanılan ve en temel kaotik denklem **Lojistik Harita**'dır:

$$x_{n+1} = r \cdot x_n \cdot (1 - x_n)$$

- $x_n \in (0, 1)$ : Sistemin  $n$ . adımdaki mevcut durum değeridir. Her iterasyonda güncellenir.
- $r \in (3.57, 4)$ : Çatallanma (bifurcation) veya büyüme parametresidir. Sistemin kaotik rejimde çalışmasını sağlar ve gizli anahtarın (secret key) bir parçasıdır.
- $x_0 \in (0, 1)$ : Sistemin başlangıç değeridir (seed) ve gizli anahtarın diğer parçasıdır.

**Önemli Eşik:**  $r$  değeri 3.57'den küçük olduğunda sistem periyodik veya sabit bir değere yakınsar; ancak  $r > 3.57$  olduğunda sistem tam kaotik rejime geçer, asla kendini tekrar etmez

ve tam tahminsizlik üretir.



## 4. KAOS TABANLI AKIŞ ŞİFRELEME (STREAM CIPHER) MİMARİSİ

Kaotik bir denklemi, verileri bit bit veya byte byte şifreleyen bir akış şifreleyicisine dönüştürmek için şu adımlar izlenir:

1. **Anahtar Tanımlama:** Gönderici ve alıcı gizli anahtar olarak bir  $(x_0, r)$  ikilisi belirler.
2. **İterasyon ve Üretim:** Lojistik harita denklemi her adımda çalıştırılarak  $x_1, x_2, x_3, \dots$  şeklinde ondalıklı (floating-point) bir dizi üretilir.
3. **Byte Dönüşümü (Quantization):** Elde edilen ondalıklı değerler, tam sayı formatında birer byte'a indirgenir:

$$k_n = \lfloor x_n \times 256 \rfloor$$

4. **Keystream Oluşumu:**  $k_1, k_2, k_3, \dots$  değerleri bizim **keystream (anahtar akışımız)** olur.
5. **Şifreleme / Şifre Çözme:** Plaintext (düz metin) bu akış ile XOR'lanarak ciphertext üretilir; çözme işlemi için ciphertext aynı akışla tekrar XOR'lanır:
  - **Şifreleme:** Plaintext  $\oplus$  Keystream = Ciphertext
  - **Şifre Çözme:** Ciphertext  $\oplus$  Keystream = Plaintext

## Metin Şifreleme Sayısal Örneği ("DATA" Kelimesi)

"DATA" kelimesini kaotik akış ile şifreleme adımları:

- **Düz Metin (Plaintext):** D (68), A (65), T (84), A (65)
- **Üretilen Kaotik Keystream:** 201, 17, 92, 244
- **XOR İşlemi ve Şifreli Metin (Ciphertext):**

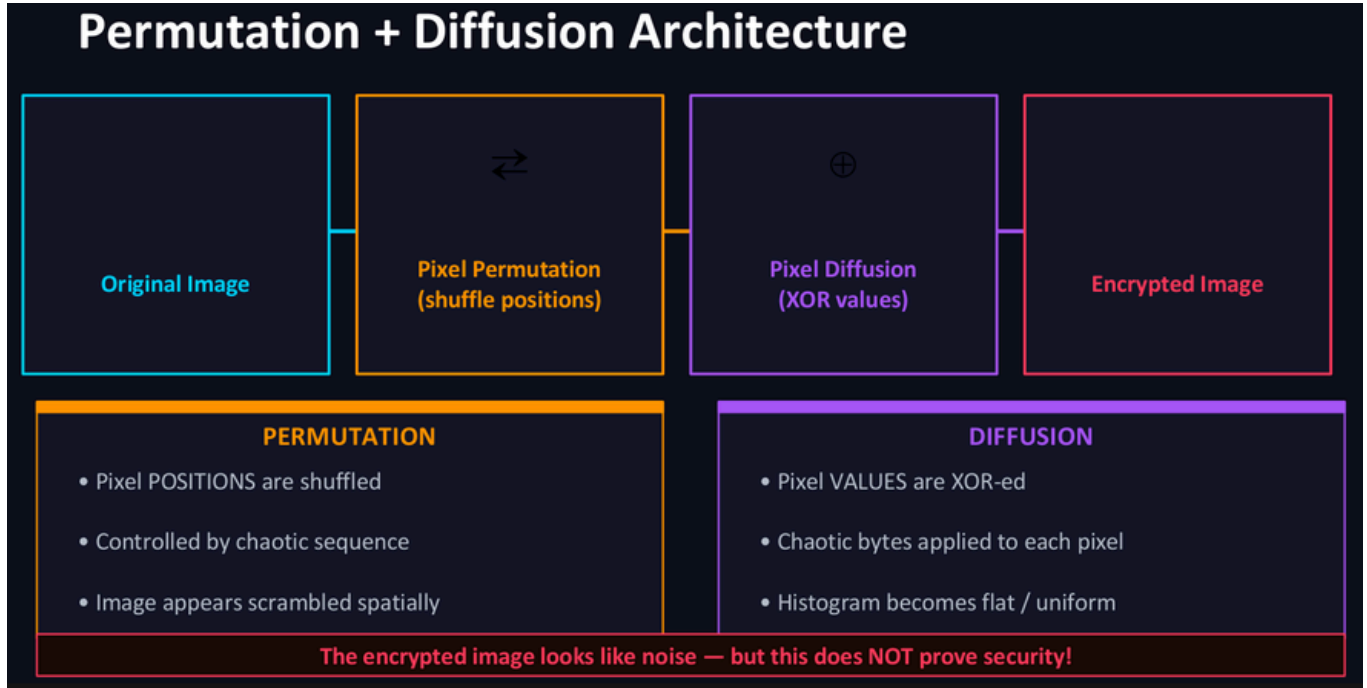
- $68 \oplus 201 = 141$
- $65 \oplus 17 = 80$
- $84 \oplus 92 = 8$
- $65 \oplus 244 = 181$
- **Çıktı (Ciphertext):** (141, 80, 8, 181)

## 5. GÖRÜNTÜ ŞİFRELEMEDE KAOS (IMAGE ENCRYPTION)

Kaotik görüntüler şifrelenirken genellikle iki aşamalı bir mimari kullanılır:

1. **Permütasyon (Permutation - Karıştırma):** Piksellerin renk değerleri değiştirilmez, ancak kaotik diziden gelen indekslere göre piksellerin resim içindeki **konumları (pozisyonları)** karıştırılır. Resim mekansal olarak tanınmaz hale gelir.
2. **Difüzyon (Diffusion - Yayma):** Piksellerin **renk değerleri**, üretilen kaotik byte'lar ile ardışık olarak XOR işlemine sokulur. Bu aşamadan sonra resmin renk histogramı tamamen düz (uniform) ve homojen bir gürültü halini alır.

**Kritik Yanılgı:** Şifrelenmiş resmin görsel olarak tamamen beyaz gürültüye (parazite) benzemesi, o sistemin güvenli olduğunu kanıtlamaz.



## 6. KAOTİK KRİPTOGRAFİNİN BEŞ BÜYÜK PROBLEMİ (Neden Üretimde Kullanılmaz?)

### Problem 1: Sonlu Hassasiyet Problemi (Finite Precision Problem)

- **Teorik Matematikte ( $\mathbb{R}$ ):** Başlangıç değeri  $x_0$  reel sayılar kümesindedir, sonsuz basamağa sahiptir, kaotik döngü sonsuza kadar asla kendini tekrar etmeden devam eder ve teorik

anahtar uzayı sonsuzdur.

- **Bilgisayar Donanımında (64-bit Kayar Nokta - IEEE 754):** Bilgisayarlar sonsuz hassasiyeti temsil edemez. Çift duyarlıklı (double) sayılar yaklaşık 15-16 anlamlı basamakla (52 bit mantissa) sınırlıdır.
- **Kriptografik Etkisi:** Bu basamak sınırlaması yüzünden oluşan yuvarlama hataları (quantization noise), matematiksel kaotik yapıyı zamanla bozar. Dizi bir süre sonra kaçınılmaz olarak **periyodik bir döngüye girer** ve kendini tekrar etmeye başlar.

## Problem 2: Anahtar Uzayı İllüzyonu (Key Space Illusion)

Birçok akademik makale şu iddiada bulunur:  $x_0 \in (0, 1)$  ve  $r \in (3.57, 4)$  aralığında sonsuz reel sayı vardır, dolayısıyla anahtar uzayımız  $10^{45}$ 'tir.

- **Gerçeklik:** IEEE 754 standardına göre  $(0, 1)$  aralığında bilgisayarın ayırt edebileceği benzersiz değer sayısı sonsuz değil, yalnızca  $\approx 2^{52} \approx 4.5 \times 10^{15}$  adettir.
- Hem  $x_0$  hem de  $r$  bu donanım sınırını paylaştığı için, sistemin gerçek birleşik entropisi en fazla **90-100 bit** civarındadır ( $10^{45}$  değil!). Bu durum kaba kuvvet (brute-force) saldırılarını uygulanabilir hale getirir.

## Problem 3: Bilinen Düz Metin Saldırısı (Known-Plaintext Attack - KPA)

Kaotik sistemlerin en büyük zafiyetlerinden biri cebirsel ve denklemsel yapılarından ötürü KPA'ya karşı dayanıksız olmalarıdır.

1. Saldırgan, bilinen bir protokol sabiti veya dosya başlığı sayesinde birkaç byte düz metne ve şifreli metne sahip olur.
2.  $\text{Plaintext}[i] \oplus \text{Ciphertext}[i]$  işlemiyle keystream byte'larını ( $k_n$ ) kolayca çıkarır.
3. Elde edilen byte'lardan  $x_n \approx k_n/256$  yaklaşımı yapılır. Lojistik harita denklemi tersine mühendisliğe açık olduğu için ardışık iki  $x$  değeri yardımıyla büyüme parametresi  $r$  kolayca kurtarılabilir:

$$r \approx \frac{x_{n+1}}{x_n \cdot (1 - x_n)}$$

4. Parametreler  $(x_0, r)$  bir kez deşifre edildiğinde, saldırgan tüm akışı yeniden üreterek mesajın tamamını çözer.

## Problem 4: İstatistiksel Testlerin Yetersizliği

- Kaotik bir şifrenin çıktısı uniform histogram, yüksek entropi ( $\approx 8$  bit/byte) ve düşük piksel korelasyonu testlerinden (NIST, NPCR, UACI) tam puan alabilir.
- Ancak bu testler sadece çıktının "rastgele görünüp görünmediğini" ölçer; saldırganın sisteme dönük geliştireceği matematiksel saldırılara (Key Recovery, Chosen-Plaintext

Attack) karşı direnci ölçemez.

## Problem 5: Gizlilikle Gelen Güvenlik Yanılgısı (Security by Obscurity)

- **Kerckhoffs Prensipleri (1883):** Bir kriptosistemin güvenliği, algoritmanın gizliliğine değil, tamamen anahtarın gizliliğine dayanmalıdır. Algoritma tamamen internete ifşa olsa bile sistem güvenli kalmalıdır.
- Kaotik tasarımlar genellikle harita türlerini veya denklemlerini gizli tutarak (proprietary) sahte bir güvenlik algısı yaratırlar. AES ise 2001'den beri tamamen açık kaynaktır ve 20 yılı aşkın süredir tüm dünyanın ortak kriptanaliz saldırılarına direnmiştir.

## 7. MODERN KRİPTO VS KAOTİK TASARIMLAR KARŞILAŞTIRMA TABLOSU

| Kriter           | Standart Kriptografi (AES / ChaCha20 / Ascon)                 | Kaos Tabanlı Şifreleyiciler                                |
|------------------|---------------------------------------------------------------|------------------------------------------------------------|
| Standardizasyon  | NIST Sertifikalı (FIPS 197, SP 800-232)                       | Genellikle standardı yok, deneysel                         |
| Açık Kriptanaliz | 20+ yıl boyunca tüm küresel saldırılara dayandı               | Sınırlı; önerilen modellerin çoğu hızlıca kırıldı          |
| Güvenlik Modeli  | Formal Güvenlik İspatları (IND-CPA, IND-CCA)                  | Çoğu zaman matematiksel güvenlik modeli tanımsız           |
| Test Yöntemi     | Güçlü Saldırı Modelleri + Formal Analiz                       | Ağırlıklı olarak yüzeysel istatistiksel testler            |
| Sonlu Hassasiyet | Doğrudan dijital/bilişim mimarileri için tasarlandı           | Sayısal donanımlarda ciddi bozulma ve periyot riski var    |
| Anahtar Uzayı    | AES-256 için net ve garantili $2^{256}$ kombinasyon           | Float kısıtlamaları yüzünden makalelerde abartılı/aldatıcı |
| KPA Direnci      | Confusion + Diffusion (Geri hesaplanabilir tek yönlü bağ yok) | Denklem yapısı gereği parametre kurtarma mümkün            |
| Kullanım Alanı   | TLS, Disk Şifreleme, VPN, Bankacılık Mimarileri               | Sadece akademik araştırma ve laboratuvar projeleri         |

## 8. GÜVENLİ ALTERNATİF: ASCON SUITE (NIST SP 800-232)

Kaotik şifrelerin hedeflediği IoT (Nesnelerin İnterneti) ve kısıtlı kaynaklara sahip gömülü sistemler için NIST, resmi olarak **Ascon** algoritma ailesini standartlaştırmıştır:

- **Ascon-AEAD128:** Kısıtlı donanımlar için kimlik doğrulamalı şifreleme (authenticated encryption) sağlar.
- **Ascon-Hash256:** Güvenli kriptografik özet (hashing) fonksiyonudur.

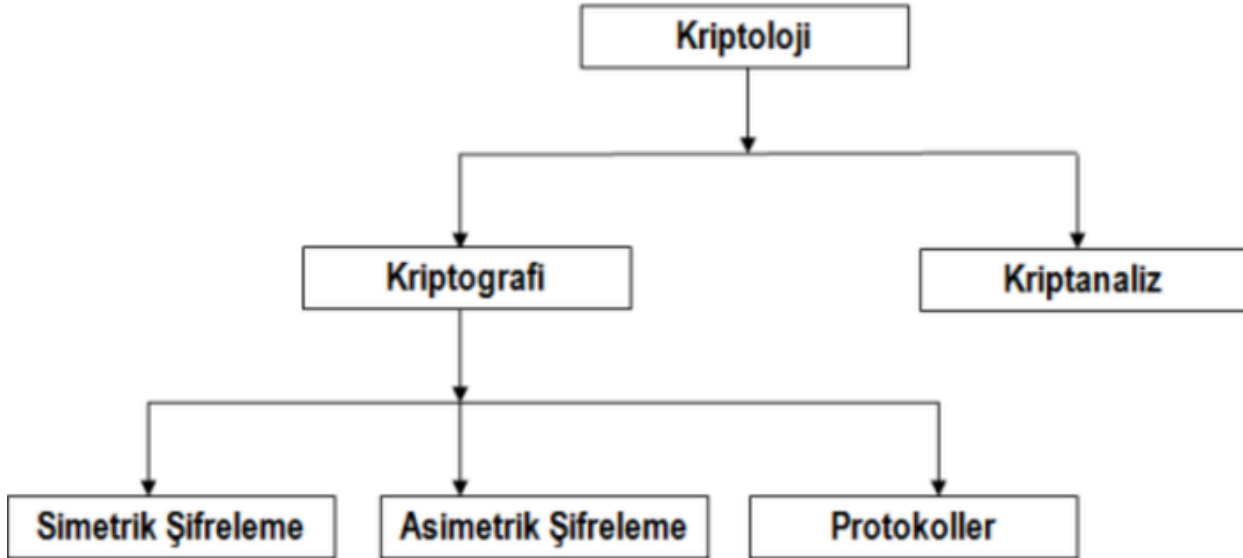
- **Ascon-XOF128 & Ascon-CXOF128:** Uzatılabilir ve özelleştirilebilir çıktı fonksiyonları varyantlarıdır.

**En Büyük Fark:** Ascon, donanımsal kısıtlamalardan etkilenmeyen formal matematiksel ispatlara sahiptir ve kaos şifrelerinin aksine tüm dünyada kabul gören bir standarttır.

## W12 ÇOK-BOYUTLU KAOTİK SİRENKLİ GÖRÜNTÜ ŞİFRELEME YÖNTEMİ

### 1. KRİPTOLOJİYE GİRİŞ VE TEMEL KAVRAMLAR

- Kriptoloji temelde ikiye ayrılır: Kriptografi (şifreleme bilimi) ve Kriptanaliz (şifre kırma bilimi).
- Kriptografi de kendi içinde Simetrik, Asimetrik ve Protokoller olarak ayrılır.
- **Simetrik Şifreleme:** Şifrelemek ve çözmek için aynı anahtar kullanılır.
- **Asimetrik Şifreleme:** Biri Public (Açık), diğeri Secret (Gizli) olmak üzere iki farklı anahtar kullanılır.
- Genelde AES, DES, IDEA, RSA gibi klasik algoritmalar **Blok Şifreleme** kullanır.
- Kablosuz ağlarda (WIFI) WEP şifrelemesinde kullanılan RC4 ise bir **Akış (Stream) Şifreleme** örneğidir.



### 2. KAOS TEORİSİ VE KAOTİK HARİTALAR (MAPS)

Klasik algoritmalara (AES, DES) alternatif olarak **Doğrusal Olmayan Dinamik Sistemlere Dayalı Yöntemler (Kaos)** geliştirilmiştir.

- Kaos teorisi; fizik, tıp, biyoloji gibi alanlarda kullanılan ve "başlangıç koşullarına oldukça duyarlı olan" dinamik sistemleri inceler.
- Başlangıç koşullarındaki milimetrik bir değişim, uzun vadede sistemde çok büyük değişikliklere (kelebek etkisi) sebep olur ve tahmin edilemez sonuçlar doğurur.

- Bu sistemler deterministik (kurallı/belirlenimci) olmalarına karşın dışarıdan tahmin edilemezler.

## A. Lojistik Harita (Logistic Map)

- 2. dereceden polinomsal bir haritadır.
- Basit doğrusal olmayan (non-linear) eşitliklerin bile kaotik davranışlar sergileyebileceğinin en net örneğidir.
- Ayrık zamanlı eşitliği:  $x_{t+1} = f_r(x) = r \cdot x_t(1 - x_t)$  formülü ile ifade edilir.
- Burada başlangıç koşulu  $x_0 \in [0, 1]$  aralığında, kontrol parametresi ise  $0 < r < 4$  aralığında seçilir.

## B. Baker Harita (Baker Map) ve İyileştirilmiş Sistemler

- Baker Harita, birim kareden içe doğru elde edilen kaotik bir harita çeşididir.
- $\alpha$  isimli bir kontrol parametresi vardır ve bu değer  $(0, 1)$  aralığında olduğunda sistem kaotik sonuçlar üretir.
- **İyileştirilmiş Sistemler:** Mevcut haritaların güvenliğini artırmak için ekstra doğrusal fonksiyonlar (örneğin  $h(x_{i-1}) = b_1x_{i-1} + b_2$ ) eklenerek daha karmaşık  $FL()$  fonksiyonları elde edilir.

$$(x_n, y_n) = \begin{cases} \left( \frac{x_{n-1}}{\alpha}, \alpha y_{n-1} \right) & 0 < x_{n-1} \leq \alpha \\ \left( \frac{x_{n-1} - \alpha}{1 - \alpha}, (1 - \alpha)y_{n-1} + \alpha \right) & \alpha < x_{n-1} \leq 1 \end{cases}$$

$\alpha$  bir kontrol parametresi ve  $(0, 1)$  aralığında ise kaotik karakterli sonuçlar üretebilir

## 3. KAOTİK GÖRÜNTÜ ŞİFRELEME ALGORİTMASI

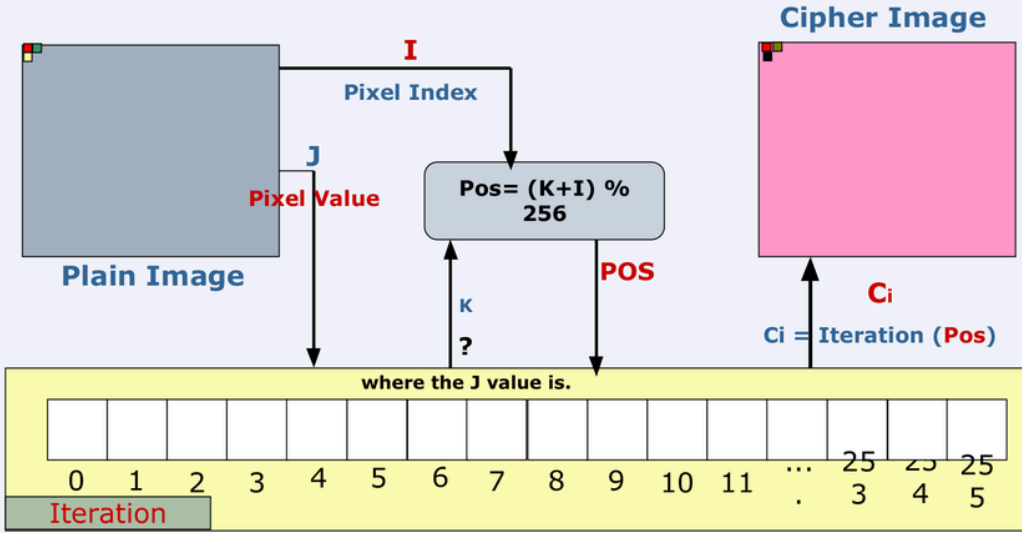
Kaotik bir kriptosistem temelde iki genel aşamadan oluşur: 1) Rastgele sayı üretmek. 2) Bu sayılar ile veriyi şifrelemek.

### Şifreleme Adımları:

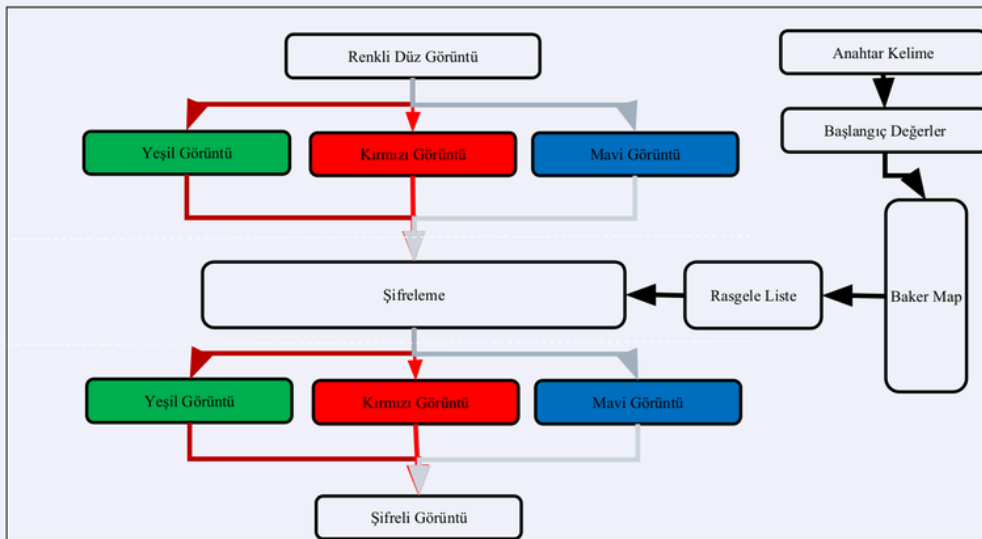
- Görüntüdeki tüm piksel değerleri  $P$  isimli tek boyutlu bir listeye/diziye yerleştirilir ( $P = \{P_0, P_1, \dots, P_{m*n}\}$ ). Burada  $m$  ve  $n$  görüntünün boyutlarıdır.

- Şifreli pikseller ( $C_i$ ) şu mantıkla hesaplanır:  $Pos = (i + k) \pmod{256}$  ve  $C_i = Iteration(Pos)$ .
- Yani pikselin indeksi ( $I$ ) ve kaotik akıştan gelen bir değer ( $K$ ) toplanıp mod 256'sı alınarak yeni bir pozisyon elde edilir.
- **Renkli Görüntü Şifreleme:** Renkli bir görüntü doğrudan şifrelenmez. Önce RGB kanallarına (Kırmızı, Yeşil, Mavi) ayrılır. Her bir renk kanalı ayrı ayrı Baker/Lojistik harita ile şifrelenir ve en son birleştirilerek nihai renkli şifreli görüntü elde edilir.

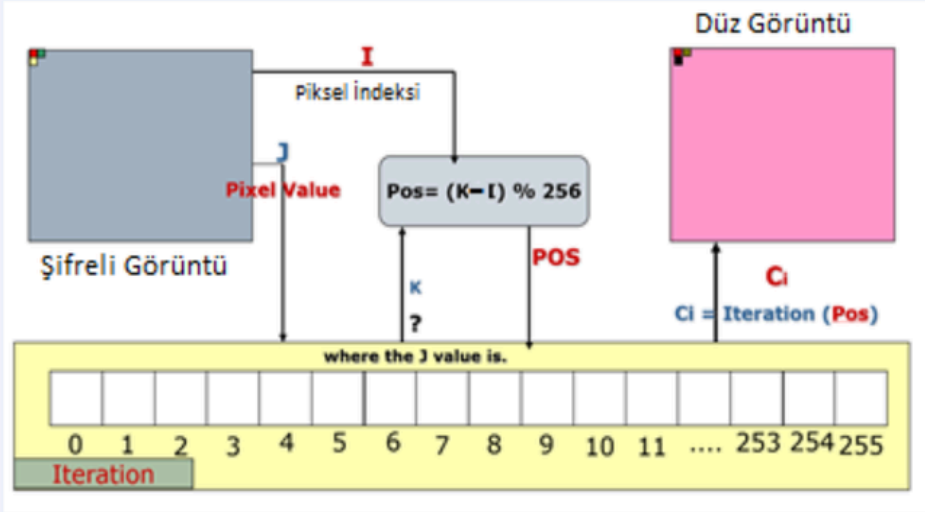
## Encryption



## Şifreleme Genel Yöntemi



# Deşifreleme



## 4. GÜVENLİK ANALİZLERİ (SINAV İÇİN EN KRİTİK BÖLÜM)

Hocalar bu kısımdaki analizlerin ne anlama geldiğini sormayı çok sever. Düz bir metin (veya görüntü) şifrelendiğinde, gerçekten güvenli olup olmadığını kanıtlamak için aşağıdaki 4 teste sokulur.

### A. İki Bitişik Pikselin Korelasyonu

- Normal (Düz) bir fotoğrafta yan yana duran piksellerin renkleri genelde birbirine çok benzerdir. Yani aralarındaki "Korelasyon" (ilişki) 1'e çok yakındır (Örn: 0.9883).
- Başarılı bir şifreleme algoritması, pikselleri öyle bir karıştırmalıdır ki yan yana duran iki pikselin birbiriyle hiçbir ilişkisi kalmamalıdır.
- Şifreli görüntüde Dikey, Yatay ve Çapraz korelasyon değerleri ölçüldüğünde bu değerlerin **0'a çok yakın** (Örn: 0.0049 veya 0.0051) olması sistemin başarılı olduğunu kanıtlar.

### B. Diferansiyel Analiz (NPCR ve UACI)

Saldırganlar, düz metinde sadece 1 pikseli değiştirip şifreli metindeki değişimi gözlemleyerek anahtarı bulmaya çalışırlar (Diferansiyel saldırı). Buna karşı iki ölçüm yapılır:

- **NPCR (Piksel Sayısı Değişim Hızı):** Girdideki orijinal görüntüde sadece bir pikseli değiştirdiğimizde, şifrelenmiş çıktı görüntüsündeki piksellerin yüzde kaçının değiştiğini ölçer. İdeal ve geçme sınırı **%99.6** civarındır.
- **UACI (Entegre Ortalama Değişim Yoğunluğu):** İki şifreli görüntü (biri orijinalden, diğeri 1 pikseli değiştirilmiş orijinalden üretilen) arasındaki ortalama piksel yoğunluğu farkını ölçer. İdeal ve geçme sınırı **%33.4** civarındır.

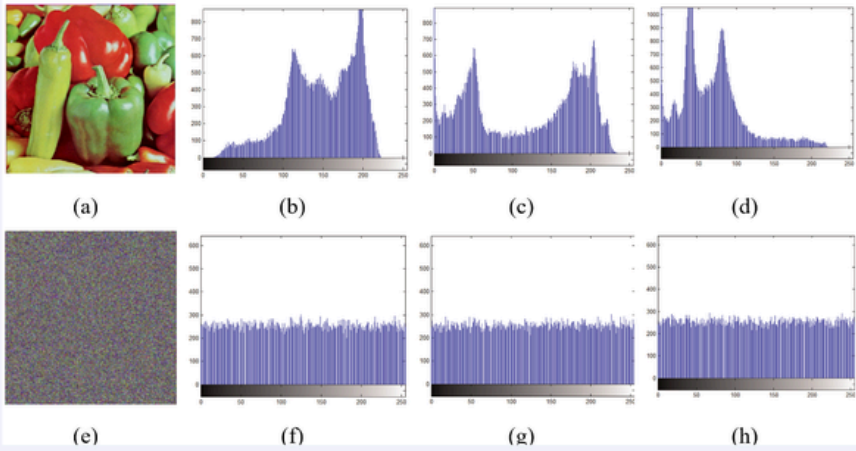
## C. Histogram Analizi

- Bir görüntünün histogramı, o görüntüde hangi renkten kaç tane olduğunu gösteren grafikdir.
- Normal bir görüntünün (Örn: Biber veya Lena resmi) R, G, B histogramları inişli çıkışlı, dağlık tepelik yapıdadır.
- Şifrelenmiş görüntünün R, G, B histogramları ise tamamen **düz (uniform)** olmalıdır. Bu, her piksel değerinden eşit sayıda var gibi görünmesini sağlar ve saldırırganın frekans analizi yapmasını engeller.

## D. Bilgi Entropisi (Information Entropy)

- Sistemin ve piksellerin "Rastgelelik" derecesini ölçer.
- 8-bitlik gri seviyeli veya ayrılmış RGB bir kanal için teorik olarak ulaşılabilir en mükemmel entropi (tam rastgelelik) değeri **8'dir**.
- Önerilen kaotik yöntemler genelde **7.99** gibi 8'e çok yakın muazzam değerler vererek sistemin bilgi sızdırmadığını (tam rastgele olduğunu) ispatlar.

## Güvenlik Analizi(Histogram Analizi )



(a) Düz Görüntü, (b) Düz Görüntünün R Histogramı, (c) Düz Görüntünün G Histogramı, (d) B Düz Görüntünün Histogramı, (e) Şifreli Görüntü, (f) R Şifreli Görüntünün Histogramı, (g) G CIPHER Görüntüsünün Histogramı, (h) B CIPHER Görüntüsünün Histogramı,